



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-04: Clases Objetos y Métodos
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	1 - B
Alumnos participantes:	Núñez Espin Bryan Sebastián
Asignatura:	Algoritmos y Lógica de Programación.
Docente:	Ing. CAIZA CAIZABUANO JOSE RUBEN, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas utilizando clases, objetos y métodos, lo cual permite disminuir errores reutilizar el código.

Específicos:

- Diseñar e implementar la clase Estudiante en C++ y Java utilizando encapsulamiento (atributos privados) y métodos de acceso (get y set).
- Construir algoritmos de validación iterativa para asegurar que las notas ingresadas por el usuario se encuentren estrictamente en el rango de 0 a 10.
- Estructurar el ciclo de vida del desarrollo de software mediante el uso de Git y GitHub, realizando commits organizados para cada etapa del proyecto.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Leer detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrollar lo solicitado.
- Codificar en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Inteligencia artificial, TAC



- Inteligencia artificial, TAC - Computador o laptop. - Plataforma educativa de la UTA.
- Internet para consultas. - Papel - Material de escritorio

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

1. Diseño y codificación de la clase Estudiante y la lógica de control en C++.
2. Diseño y codificación de la clase Estudiante y la clase principal ejecutable en Java. Implementación de mecanismos de control de errores y validación de notas en ambos lenguajes.
3. Creación de la estructura de carpetas requerida (Cpp/, Java/, capturas/).
4. Inicialización de Git, indexación de archivos y ejecución de un mínimo de 3 commits significativos.

2.7 Resultados obtenidos

Para dar solución al sistema básico de control de estudiantes y calificaciones, se estructuró el desarrollo en base a un análisis previo del problema, el diseño del algoritmo en pseudocódigo y su posterior validación lógica mediante una prueba de escritorio.

1. Análisis del Problema

- **Entradas:** Cédula, Nombre, Apellido y 3 notas parciales individuales por cada estudiante.
- **Restricciones / Validaciones:** Las notas ingresadas deben pertenecer estrictamente al rango de $[0, 10]$. Si el usuario ingresa un valor fuera de este rango, el sistema debe exigir la corrección inmediata.
- **Procesos:** Calcular el promedio aritmético simple de las tres notas:

$$Promedio = \frac{nota1 + nota2 + nota3}{3}$$

- **Determinar el estado del estudiante:** Si el $Promedio \geq 7$, el estado es "Aprobado"; de lo contrario, será "Reprobado".
- Contabilizar de manera global la cantidad total de alumnos aprobados y reprobados del grupo.
- **Salidas:** Listado tabular con toda la información de los estudiantes (Cédula, Nombre, Apellido, Notas, Promedio y Estado) y las estadísticas globales de aprobación del aula.

2. Pseudocódigo



```
Algoritmo Sistema_Control_Estudiantes
// Definición de variables globales y vectores para almacenar la información
Definir cedulas, nombres, apellidos Como Cadena
Definir nota1, nota2, nota3, promedio Como Real
Definir estados Como Cadena
Definir i, aprobados, reprobados Como Entero

Dimension cedulas[5], nombres[5], apellidos[5], estados[5]
Dimension nota1[5], nota2[5], nota3[5], promedio[5]

aprobados <- 0
reprobados <- 0

Escribir "=== INGRESO DE DATOS DE ESTUDIANTES ==="
Para i <- 1 Hasta 5 Con Paso 1 Hacer
    Escribir "Estudiante #", i
    Escribir "Ingrese Cédula:"
    Leer cedulas[i]
    Escribir "Ingrese Nombre:"
    Leer nombres[i]
    Escribir "Ingrese Apellido:"
    Leer apellidos[i]

// Validación interactiva de la Nota 1
Repetir
    Escribir "Ingrese Nota 1 (0-10):"
    Leer nota1[i]
    Si nota1[i] < 0 O nota1[i] > 10 Entonces
        Escribir "Error: La nota debe estar entre 0 y 10."
    FinSi
Hasta Que nota1[i] >= 0 Y nota1[i] <= 10

// Validación interactiva de la Nota 2
Repetir
    Escribir "Ingrese Nota 2 (0-10):"
    Leer nota2[i]
    Si nota2[i] < 0 O nota2[i] > 10 Entonces
        Escribir "Error: La nota debe estar entre 0 y 10."
    FinSi
Hasta Que nota2[i] >= 0 Y nota2[i] <= 10

// Validación interactiva de la Nota 3
Repetir
    Escribir "Ingrese Nota 3 (0-10):"
    Leer nota3[i]
    Si nota3[i] < 0 O nota3[i] > 10 Entonces
        Escribir "Error: La nota debe estar entre 0 y 10."
    FinSi
Hasta Que nota3[i] >= 0 Y nota3[i] <= 10
```



```
// Procesamiento de cálculos internos
promedio[i] <- (nota1[i] + nota2[i] + nota3[i]) / 3.0

Si promedio[i] >= 7.00 Entonces
    estados[i] <- "Aprobado"
    aprobados <- aprobados + 1
Sino
    estados[i] <- "Reprobado"
    reprobados <- reprobados + 1
FinSi
Escribir "-----"
FinPara

// Despliegue de los resultados obtenidos
Escribir ""
Escribir "===== REPORTE
GENERAL DE NOTAS ====="
Escribir "Cédula | Nombre | Apellido | Nota 1 | Nota 2 | Nota 3 |
Promedio | Estado"
Para i <- 1 Hasta 5 Con Paso 1 Hacer
    Escribir cedulas[i], " | ", nombres[i], " | ", apellidos[i], " | ", nota1[i], " | ",
nota2[i], " | ", nota3[i], " | ", promedio[i], " | ", estados[i]
FinPara
Escribir
"=====
=====

Escribir "=== INDICADORES GLOBALES ==="
Escribir "Estudiantes Aprobados: ", aprobados
Escribir "Estudiantes Reprobados: ", reprobados
FinAlgoritmo
```

3. Listado de Archivos a Utilizar

Para cumplir rigurosamente con el esquema del repositorio y los entregables de la guía práctica, el proyecto requiere el desarrollo y la gestión de los siguientes archivos específicos:

- Cpp/main.cpp: Archivo fuente que contiene la definición de la clase Estudiante en C++, funciones de validación de entradas y la función conductora principal `main()`.
- Java/Estudiante.java: Archivo de definición de clase en Java bajo el principio de encapsulamiento.
- Java/Main.java: Archivo de control lógico en Java que gestiona las capturas de teclado, el llenado de colecciones dinámicas y la muestra de estadísticas finales en consola.
- README.md: Archivo de documentación técnica en formato Markdown para la visualización inicial de la raíz del repositorio en GitHub.



4. Pruebas de Escritorio

Para garantizar el correcto comportamiento del algoritmo antes del despliegue del software, se planteó una prueba de escritorio con valores controlados, incluyendo un caso donde el sistema debe rechazar una nota inválida y forzar el reingreso.

Tabla 1: Trazo analítica de la prueba de escritorio

Iteración (i)	Cédula	Nombre	Apellido	Nota 1	Nota 2	Nota 3	Entrada Procesada	Promedio Calculado	Estado Asignado
1	185012	Juan	Pérez	8.0	9.0	7.0	Aceptadas	8.00	Aprobado
2	172345	Maria	López	5.0	6.0	4.0	Rechazada	5.00	Reprobado
3	098765	Carlos	Mina	10.0	10.0	9.5	Aceptadas	9.83	Aprobado
4	180421	Ana	Silva	6.5	7.0	7.5	Aceptadas	7.00	Aprobado
5	120344	Luis	Borja	4.0	5.5	6.0	Aceptadas	5.16	Reprobado

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☒ La empatía
- ☐ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

La Programación Orientada a Objetos demostró ser un paradigma eficiente para modularizar el código, ya que encapsular los datos sensibles de los estudiantes (como sus notas y datos de identidad) bajo atributos privados impide modificaciones accidentales desde el exterior de la clase. La automatización de procesos internos (como el recuento y cálculo inmediato del promedio mediante constructores) minimiza drásticamente el error humano en aplicaciones transaccionales y de gestión escolar. El uso de sistemas de control de versiones distribuidos (Git) combinado con plataformas en la nube (GitHub) proporciona un rastro transparente del ciclo de vida del software, facilitando auditorías de código mediante mensajes de commit bien estructurados.



2.10 Recomendaciones

Implementar métodos de persistencia de datos (archivos .txt o .dat) en futuras iteraciones prácticas para evitar la pérdida de información de los estudiantes al cerrar la consola de comandos. Utilizar estructuras dinámicas de datos nativas de cada lenguaje (como `std::vector` en C++ y `ArrayList` en Java) en lugar de arreglos estáticos de tamaño fijo, para permitir un escalamiento dinámico y seguro del software. Mantener la convención de nomenclatura estándar de la industria (CamelCase en Java y `snake_case` o CamelCase modular en C++) para asegurar la legibilidad del código fuente por terceros.

2.11 Referencias bibliográficas

- [1] H. M. Deitel y P. J. Deitel, *C++ Cómo programar*, 10a ed. Ciudad de México: Pearson Educación, 2018.
- [2] H. Schildt, *Java: El Manual de Referencia*, 11a ed. Nueva York: McGraw-Hill, 2019.
- [3] J. R. Caiza, "Guía Práctica APE 04 – Clases, Objetos y Métodos", Universidad Técnica de Ambato, Ambato, Ecuador, 2026.

2.12 Anexos

Diagrama de flujo



```
1. #include <iostream>
2. #include <vector>
3. #include <string>
4. #include <iomanip>
5.
6. using namespace std;
7.
8. // Clase Estudiante con atributos privados
9. class Estudiante {
10. private:
11.     string cedula;
12.     string nombre;
13.     string apellido;
14.     double nota1;
15.     double nota2;
16.     double nota3;
17.     double promedio;
18.     string estado;
19.
20. public:
21.     // Constructor de la clase
22.     Estudiante(string _cedula, string _nombre, string _apellido,
23. double _n1, double _n2, double _n3) {
24.         cedula = _cedula;
25.         nombre = _nombre;
26.         apellido = _apellido;
27.         nota1 = _n1;
28.         nota2 = _n2;
29.         nota3 = _n3;
30.         calcularPromedio();
31.         determinarEstado();
32.     }
33.
34.     // Métodos Get y Set para el acceso seguro a atributos
35.     (Encapsulamiento)
36.     string getCedula() { return cedula; }
37.     void setCedula(string _cedula) { cedula = _cedula; }
38.
39.     string getNombre() { return nombre; }
40.     void setNombre(string _nombre) { nombre = _nombre; }
41.
42.     string getApellido() { return apellido; }
43.     void setApellido(string _apellido) { apellido = _apellido; }
44.
45.     double getNota1() { return nota1; }
46.     void setNota1(double _n1) { nota1 = _n1; calcularPromedio();
47.         determinarEstado(); }
```



```
46.     double getNota2() { return nota2; }
47.     void setNota2(double _n2) { nota2 = _n2; calcularPromedio();
    determinarEstado(); }
48.
49.     double getNota3() { return nota3; }
50.     void setNota3(double _n3) { nota3 = _n3; calcularPromedio();
    determinarEstado(); }
51.
52.     double getPromedio() { return promedio; }
53.     string getEstado() { return estado; }
54.
55.     // Método interno para calcular el promedio de las 3 notas
56.     void calcularPromedio() {
57.         promedio = (nota1 + nota2 + nota3) / 3.0;
58.     }
59.
60.     // Método para determinar si aprueba o reprueba (Condición >=
    7.00)
61.     void determinarEstado() {
62.         if (promedio >= 7.00) {
63.             estado = "Aprobado";
64.         } else {
65.             estado = "Reprobado";
66.         }
67.     }
68.
69.     // Método para mostrar la información individual del estudiante
    de manera tabular
70.     void mostrarInformacion() {
71.         cout << left << setw(12) << cedula
72.             << setw(15) << nombre
73.             << setw(15) << apellido
74.             << fixed << setprecision(2)
75.             << setw(8) << nota1
76.             << setw(8) << nota2
77.             << setw(8) << nota3
78.             << setw(10) << promedio
79.             << setw(12) << estado << endl;
80.     }
81.};
82.
83.// Función modular para validar que las notas estén estrictamente
    entre 0 y 10
84.double ingresarNotaValidada(string nombreNota) {
85.    double nota;
86.    while (true) {
87.        cout << "Ingrese " << nombreNota << " (0 - 10): ";
88.        cin >> nota;
```



```
89.         if (cin.fail() || nota < 0 || nota > 10) {
90.             cin.clear();
91.             cin.ignore(10000, '\n');
92.             cout << "Nota inválida. Intente nuevamente.\n";
93.         } else {
94.             return nota;
95.         }
96.     }
97. }
98.
99. int main() {
100.     vector<Estudiante> listaEstudiantes;
101.     int aprobados = 0;
102.     int reprobados = 0;
103.
104.     cout << "=== REGISTRO DE ESTUDIANTES (C++) ===\n\n";
105.
106.     // Registrar el mínimo solicitado de 5 estudiantes
107.     for (int i = 0; i < 5; i++) {
108.         string cedula, nombre, apellido;
109.         double n1, n2, n3;
110.
111.         cout << "Estudiante #" << (i + 1) << endl;
112.         cout << "Cédula: "; cin >> cedula;
113.         cout << "Nombre: "; cin >> nombre;
114.         cout << "Apellido: "; cin >> apellido;
115.
116.         n1 = ingresarNotaValidada("Nota 1");
117.         n2 = ingresarNotaValidada("Nota 2");
118.         n3 = ingresarNotaValidada("Nota 3");
119.         cout << "-----\n";
120.
121.         Estudiante est(cedula, nombre, apellido, n1, n2, n3);
122.         listaEstudiantes.push_back(est);
123.
124.         if (est.getEstado() == "Aprobado") {
125.             aprobados++;
126.         } else {
127.             reprobados++;
128.         }
129.     }
130.
131.     // Imprimir el listado general en consola
132.     cout << "\n===== LISTA DE
ESTUDIANTES =====\n";
133.     cout << left << setw(12) << "Cédula" << setw(15) <<
"Nombre" << setw(15) << "Apellido"
```



```
134.         << setw(8) << "Nota 1" << setw(8) << "Nota 2" <<
            setw(8) << "Nota 3"
135.         << setw(10) << "Promedio" << setw(12) << "Estado" <<
            endl;
136.         cout << "-----\n";
137.
138.         for (size_t i = 0; i < listaEstudiantes.size(); i++) {
139.             listaEstudiantes[i].mostrarInformacion();
140.         }
141.         cout <<
            "=====\n";
142.
143.         // Impresión de métricas estadísticas solicitadas
144.         cout << "\n=== ESTADÍSTICAS DEL GRUPO ===" << endl;
145.         cout << "Total de estudiantes aprobados: " << aprobados
            << endl;
146.         cout << "Total de estudiantes reprobados: " << reprobados
            << endl;
147.
148.         return 0;
149.     }
```

6. Archivo main.java

```
1. package java;
2.
3. import java.util.ArrayList;
4. import java.util.Scanner;
5.
6. import java.estudiantes;
7.
8. public class main {
9.     public static void main(String[] args) {
10.         Scanner sc = new Scanner(System.in);
11.         ArrayList<estudiantes> listaEstudiantes = new
            ArrayList<>();
12.         int aprobados = 0;
13.         int reprobados = 0;
14.
15.         System.out.println("=== REGISTRO DE ESTUDIANTES (JAVA)
            ===\n");
16.
17.         // Entrada interactiva para 5 estudiantes
18.         for (int i = 0; i < 5; i++) {
19.             System.out.println("Estudiante #" + (i + 1));
20.             System.out.print("Cédula: ");
21.             String cedula = sc.next();
```



```
22.         System.out.print("Nombre: ");
23.         String nombre = sc.next();
24.         System.out.print("Apellido: ");
25.         String apellido = sc.next();
26.
27.         double n1 = ingresarNotaValidada(sc, "Nota 1");
28.         double n2 = ingresarNotaValidada(sc, "Nota 2");
29.         double n3 = ingresarNotaValidada(sc, "Nota 3");
30.         System.out.println("-----
");
31.
32.         estudiantes est = new estudiantes(cedula, nombre,
apellido, n1, n2, n3);
33.         listaEstudiantes.add(est);
34.
35.         if (est.getEstado().equals("Aprobado")) {
36.             aprobados++;
37.         } else {
38.             reprobados++;
39.         }
40.     }
41.
42.     // Presentación de la tabla consolidada
43.     System.out.println("\n=====
LISTA DE ESTUDIANTES =====");
44.     System.out.printf("%-12s %-15s %-15s %-8s %-8s %-8s %-10s
%-12s\n",
45.         "Cédula", "Nombre", "Apellido", "Nota 1", "Nota 2",
"Nota 3", "Promedio", "Estado");
46.     System.out.println("-----
-----");
47.
48.     for (estudiantes est : listaEstudiantes) {
49.         est.mostrarInformacion();
50.     }
51.     System.out.println("=====
=====");
52.
53.     // Bloque estadístico final
54.     System.out.println("\n=== ESTADÍSTICAS DEL GRUPO ===");
55.     System.out.println("Total de estudiantes aprobados: " +
aprobados);
56.     System.out.println("Total de estudiantes reprobados: " +
reprobados);
57.
58.     sc.close();
59. }
60.
```



```
61. // Lógica recursiva/iterativa de control numérico en rango
    [0,10]
62. private static double ingresarNotaValidada(Scanner sc, String
    nombreNota) {
63.     double nota;
64.     while (true) {
65.         System.out.print("Ingrese " + nombreNota + " (0 - 10):
    ");
66.         if (sc.hasNextDouble()) {
67.             nota = sc.nextDouble();
68.             if (nota >= 0 && nota <= 10) {
69.                 return nota;
70.             }
71.         } else {
72.             sc.next();
73.         }
74.         System.out.println("Nota inválida. Debe estar en el
    rango de 0 a 10.");
75.     }
76. }
77. }
```

7. Archivo estudiantes.java

```
1. package java;
2.
3. public class estudiantes {
4.     // Atributos con alcance privado
5.     private String cedula;
6.     private String nombre;
7.     private String apellido;
8.     private double nota1;
9.     private double nota2;
10.    private double nota3;
11.    private double promedio;
12.    private String estado;
13.
14.    // Constructor polimórfico por asignación directa
15.    public estudiantes(String cedula2, String nombre2, String
    apellido2, double n1, double n2, double n3) {
16.        //TODO Auto-generated constructor stub
17.    }
18.
19.    // Métodos mutadores (Getters y Setters)
20.    public String getCedula() { return cedula; }
21.    public void setCedula(String cedula) { this.cedula = cedula; }
22.
23.    public String getNombre() { return nombre; }
24.    public void setNombre(String nombre) { this.nombre = nombre; }
```

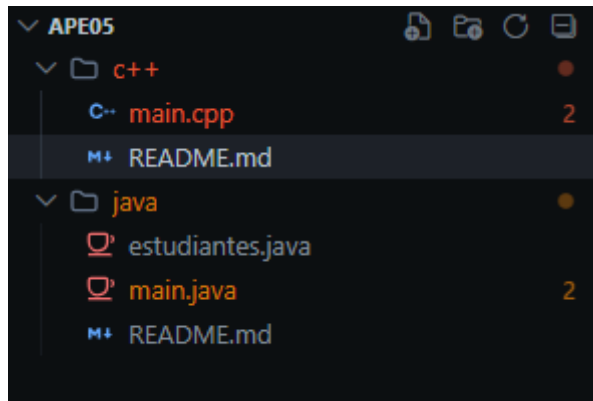


```
25.
26.     public String getApellido() { return apellido; }
27.     public void setApellido(String apellido) { this.apellido =
    apellido; }
28.
29.     public double getNota1() { return nota1; }
30.     public void setNota1(double nota1) { this.nota1 = nota1;
    calcularPromedio(); determinarEstado(); }
31.
32.     public double getNota2() { return nota2; }
33.     public void setNota2(double nota2) { this.nota2 = nota2;
    calcularPromedio(); determinarEstado(); }
34.
35.     public double getNota3() { return nota3; }
36.     public void setNota3(double nota3) { this.nota3 = nota3;
    calcularPromedio(); determinarEstado(); }
37.
38.     public double getPromedio() { return promedio; }
39.     public String getEstado() { return estado; }
40.
41.     // Calcula automáticamente la media aritmética de los
    parámetros numéricos
42.     public void calcularPromedio() {
43.         this.promedio = (this.nota1 + this.nota2 + this.nota3) /
    3.0;
44.     }
45.
46.     // Compara si el promedio se ajusta a la condición de
    aprobación base
47.     public void determinarEstado() {
48.         if (this.promedio >= 7.00) {
49.             this.estado = "Aprobado";
50.         } else {
51.             this.estado = "Reprobado";
52.         }
53.     }
54.
55.     // Despliega la información formateada en una sola fila de
    consola
56.     public void mostrarInformacion() {
57.         System.out.printf("%-12s %-15s %-15s %-8.2f %-8.2f %-8.2f
    %-10.2f %-12s\n",
58.             cedula, nombre, apellido, nota1, nota2, nota3,
    promedio, estado);
59.     }
60. }
```

8. Capturas de pantalla:



ARCHIVOS UTILIZADOS:



EJECUCION EN C++

```
Estudiante #3
Cédula: 0987654321
Nombre: Carlos
Apellido: Mina
Ingrese Nota 1 (0 - 10): 10
Ingrese Nota 2 (0 - 10): 10
Ingrese Nota 3 (0 - 10): 9.5
-----

Estudiante #4
Cédula: 1804218976
Nombre: Ana
Apellido: Silva
Ingrese Nota 1 (0 - 10): 6.5
Ingrese Nota 2 (0 - 10): 7.0
Ingrese Nota 3 (0 - 10): 7.5
-----

Estudiante #5
Cédula: 1203445612
Nombre: Luis
Apellido: Borja
Ingrese Nota 1 (0 - 10): 4.0
Ingrese Nota 2 (0 - 10): 5.5
Ingrese Nota 3 (0 - 10): 6.0
-----

===== LISTA DE ESTUDIANTES =====
Cédula      Nombre      Apellido      Nota 1  Nota 2  Nota 3  Promedio  Estado
-----
1850123456  Juan        Perez          8.50    9.00    7.50    8.33     Aprobado
1723456789  Maria       Lopez          5.00    6.00    4.00    5.00     Reprobado
0987654321  Carlos      Mina            10.00   10.00   9.50    9.83     Aprobado
1804218976  Ana         Silva          6.50    7.00    7.50    7.00     Aprobado
1203445612  Luis        Borja          4.00    5.50    6.00    5.17     Reprobado
=====

=== ESTADÍSTICAS DEL GRUPO ===
Total de estudiantes aprobados: 3
Total de estudiantes reprobados: 2

Process finished with exit code 0
```



EJECUCION EN JAVA

```
Cédula: 0987654321
Nombre: Carlos
Apellido: Mina
Ingrese Nota 1 (0 - 10): 10
Ingrese Nota 2 (0 - 10): 10
Ingrese Nota 3 (0 - 10): 9,5
-----

Estudiante #4
Cédula: 1804218976
Nombre: Ana
Apellido: Silva
Ingrese Nota 1 (0 - 10): 6,5
Ingrese Nota 2 (0 - 10): 7,0
Ingrese Nota 3 (0 - 10): 7,5
-----

Estudiante #5
Cédula: 1203445612
Nombre: Luis
Apellido: Borja
Ingrese Nota 1 (0 - 10): 4,0
Ingrese Nota 2 (0 - 10): 5,5
Ingrese Nota 3 (0 - 10): 6,0
-----

===== LISTA DE ESTUDIANTES =====
Cédula      Nombre      Apellido      Nota 1      Nota 2      Nota 3      Promedio      Estad
-----
1850123456   Juan        Perez          8,50        9,00        7,50        8,33        Aprob
1723456789   Maria       Lopez          5,00        6,00        4,00        5,00        Repro
0987654321   Carlos      Mina            10,00       10,00       9,50        9,83        Aprob
1804218976   Ana         Silva          6,50        7,00        7,50        7,00        Aprob
1203445612   Luis        Borja          4,00        5,50        6,00        5,17        Repro
=====

=== ESTADÍSTICAS DEL GRUPO ===
Total de estudiantes aprobados: 3
Total de estudiantes reprobados: 2

BUILD SUCCESSFUL (total time: 1 minute 15 seconds)
```

CAPTURA DE PANTALLA DE LOS COMMITS EN GITHUB:



9. Enlace de github:

<https://github.com/BryanNu07/APE-04>